POLITECNICO
MILANO 1863

Progetto TIW

Studente: Benedic Matteo Gruppo 16

Progetto: Specifica N°1

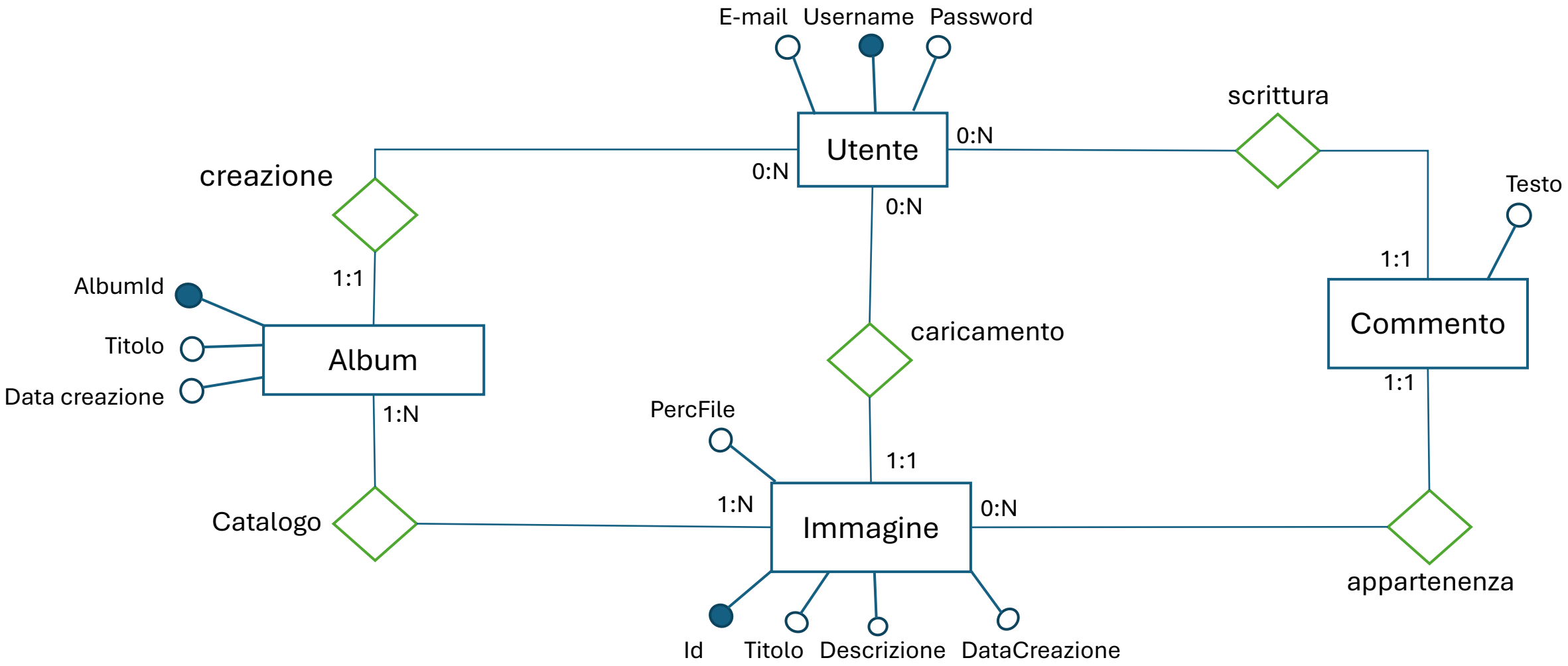
Progetto Pure HTML

Analisi dei dati

Un'applicazione web consente la gestione di una galleria d'immagini. L'applicazione supporta registrazione e login mediante una pagina pubblica con opportune form. La registrazione controlla la validità sintattica dell'indirizzo di email e l'uguaglianza tra i campi "password" e "ripeti password". La registrazione controlla l'unicità dello username. Ogni immagine è memorizzata come file nel file system del server su cui l'applicazione è rilasciata. Inoltre nella base di dati sono memorizzati i seguenti attributi: un titolo, una data di creazione, un testo descrittivo e il percorso del file dell'immagine nel file system del server. Le immagini sono associate all'utente che le carica. L'utente può creare album dalla HOME PAGE e associare a questi le proprie immagini. Un album ha un titolo, il creatore e la data di creazione. La stessa immagine può appartenere a più di un album. Le immagini sono associate a uno o più commenti inseriti dagli utenti (dal proprietario o da altri utenti). Un commento ha un testo e il nome dell'utente che lo ha creato.

- Entità, Attributi, Relazioni

Data Design


E-mail
Username
Password
Utente
0:N
0:N
0:N
scrittura
creazione
1:1
AlbumId
Titolo
Data creazione
Album
1:N
Catalogo
caricamento
1:1
PercFile
1:N
Immagine
0:N
Id
Titolo
Descrizione
DataCreazione
Testo
1:1
Commento
1:1
appartenenza

Local Database Schema

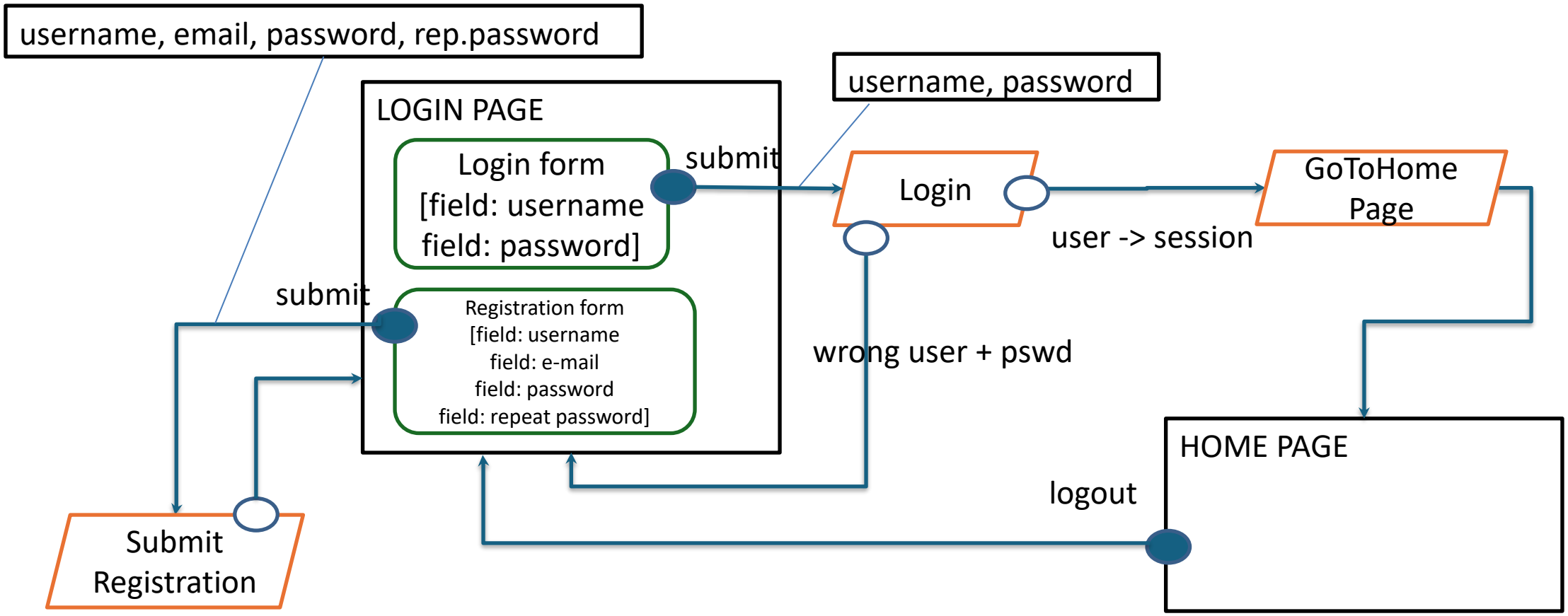
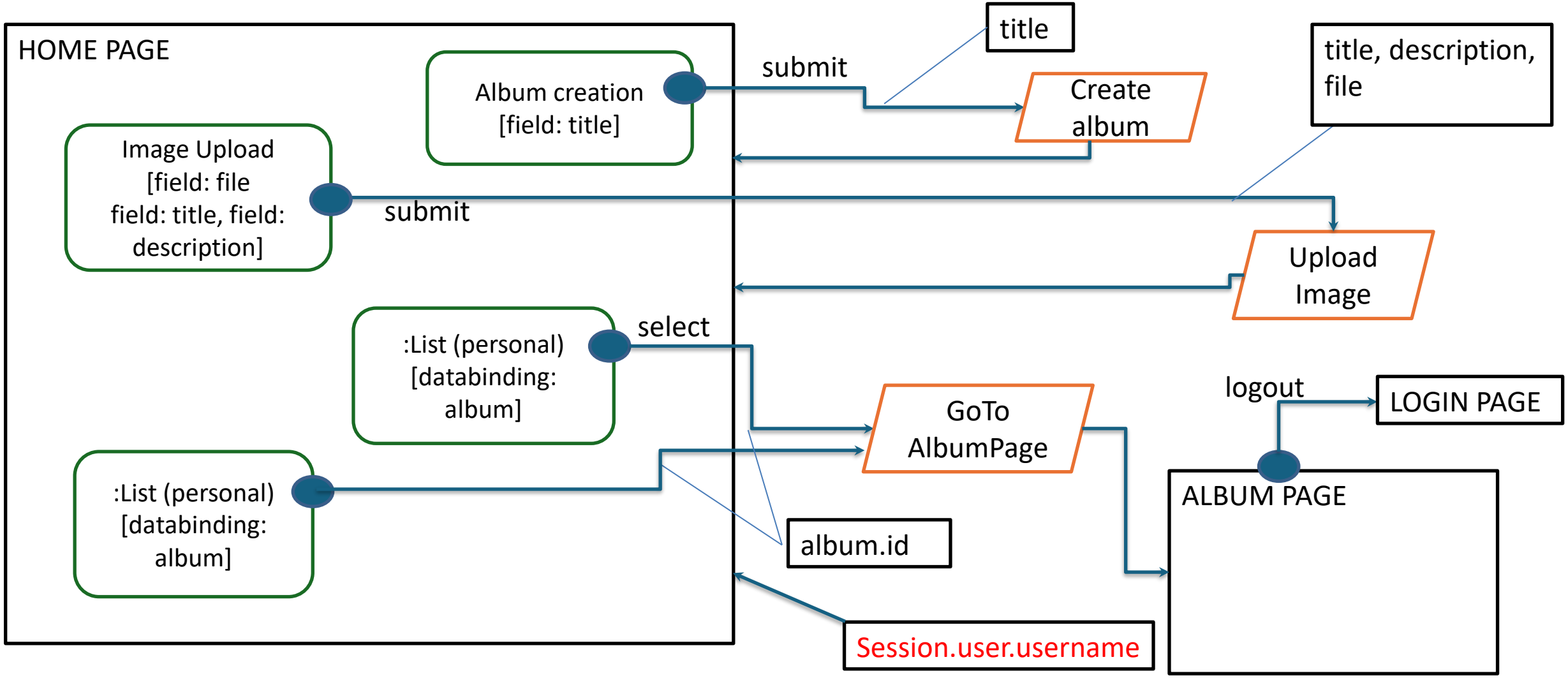
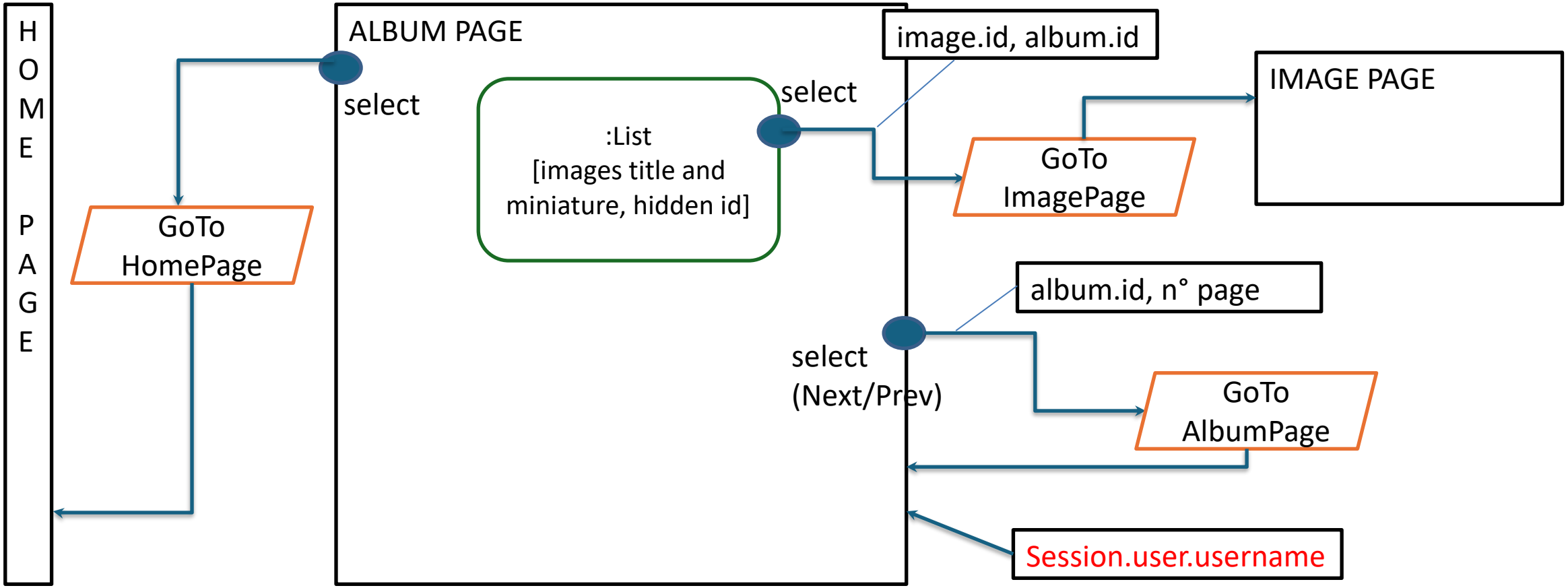
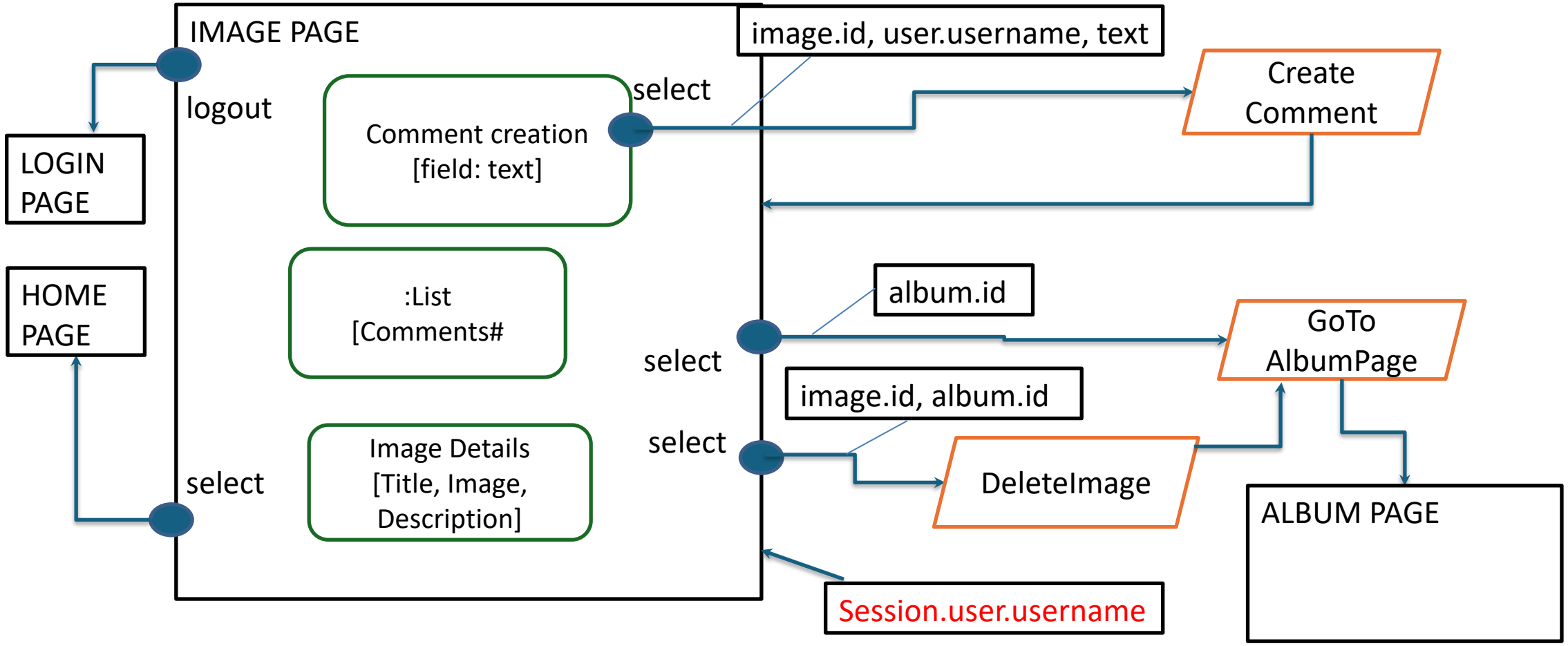
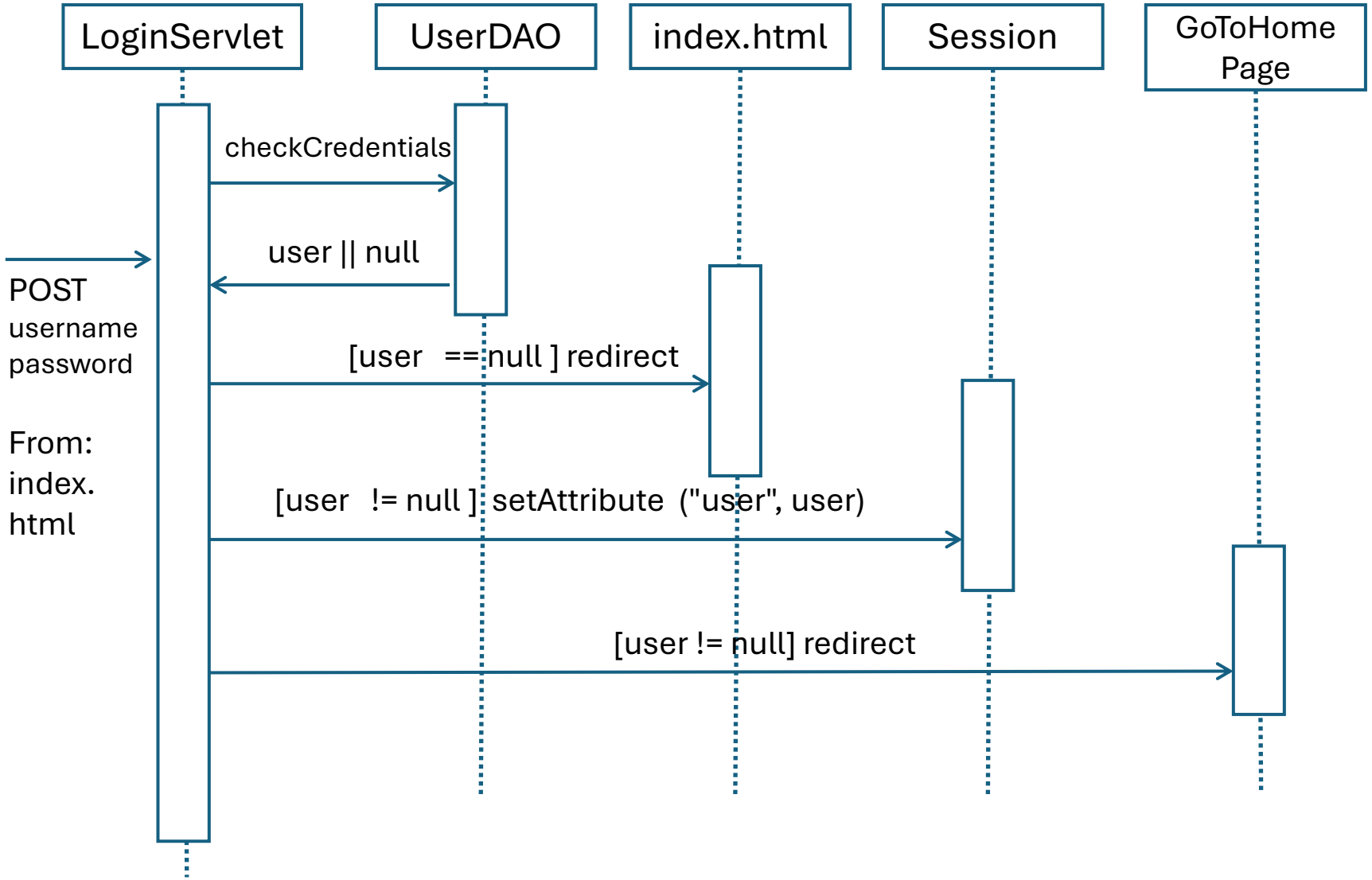
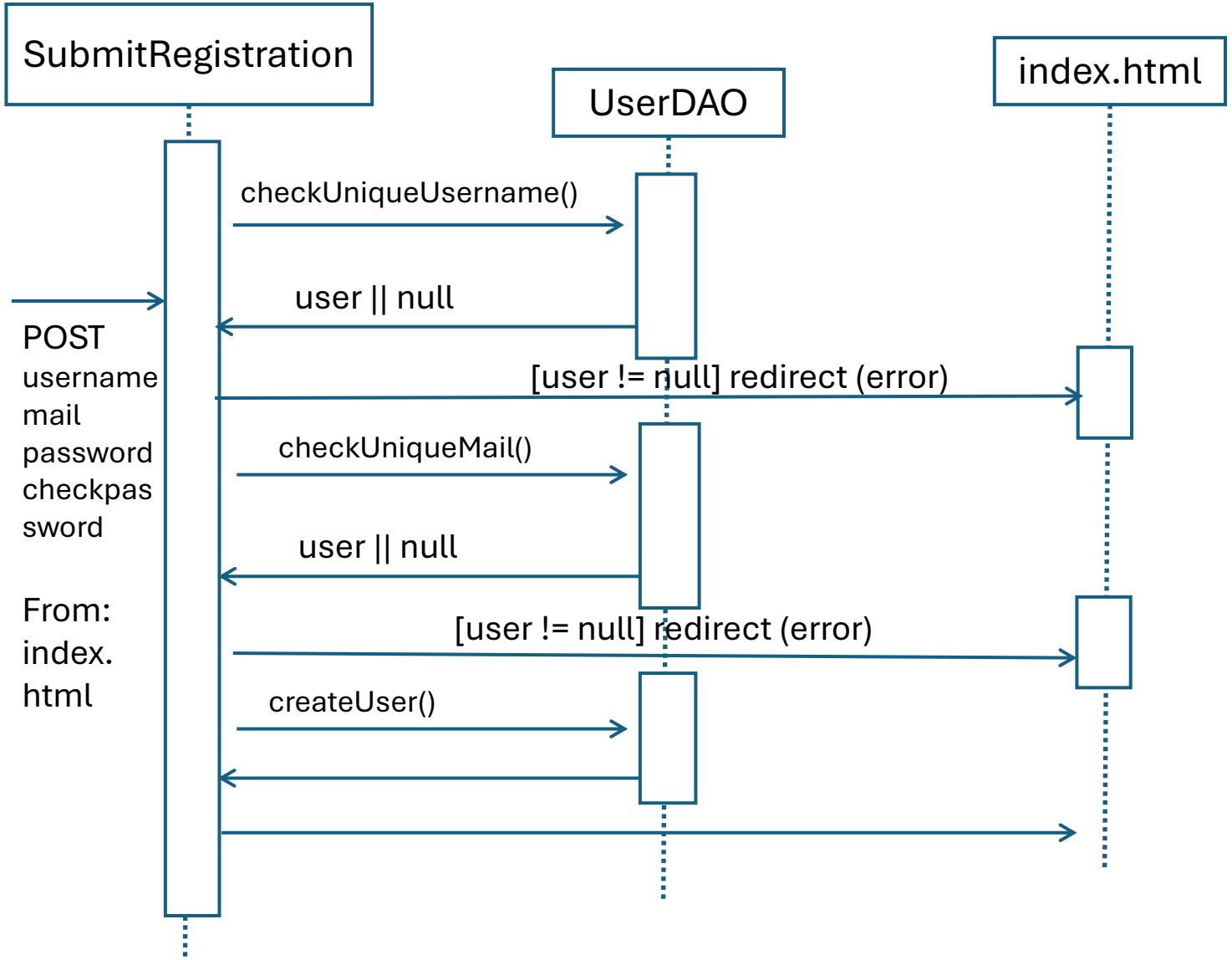
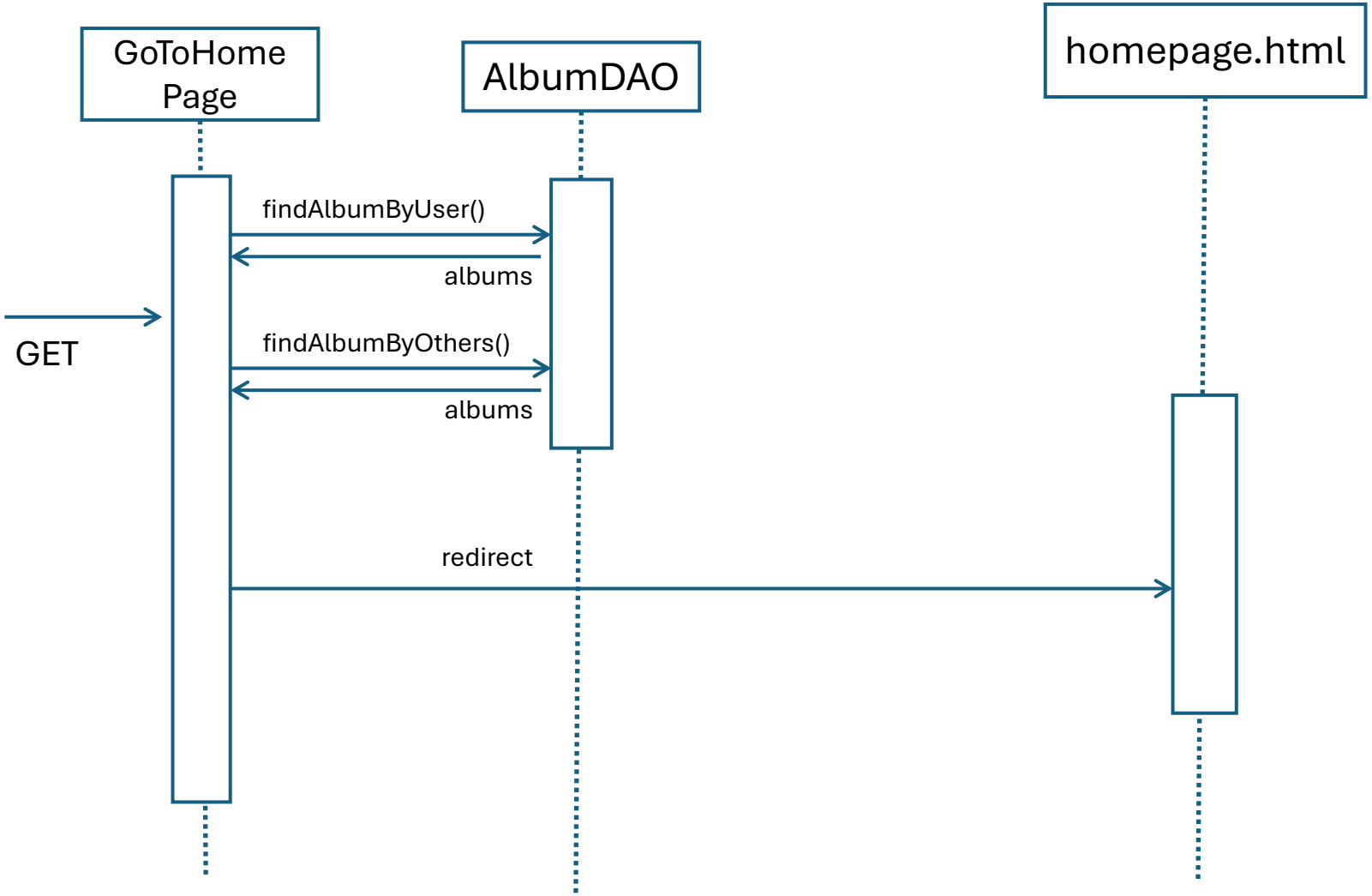
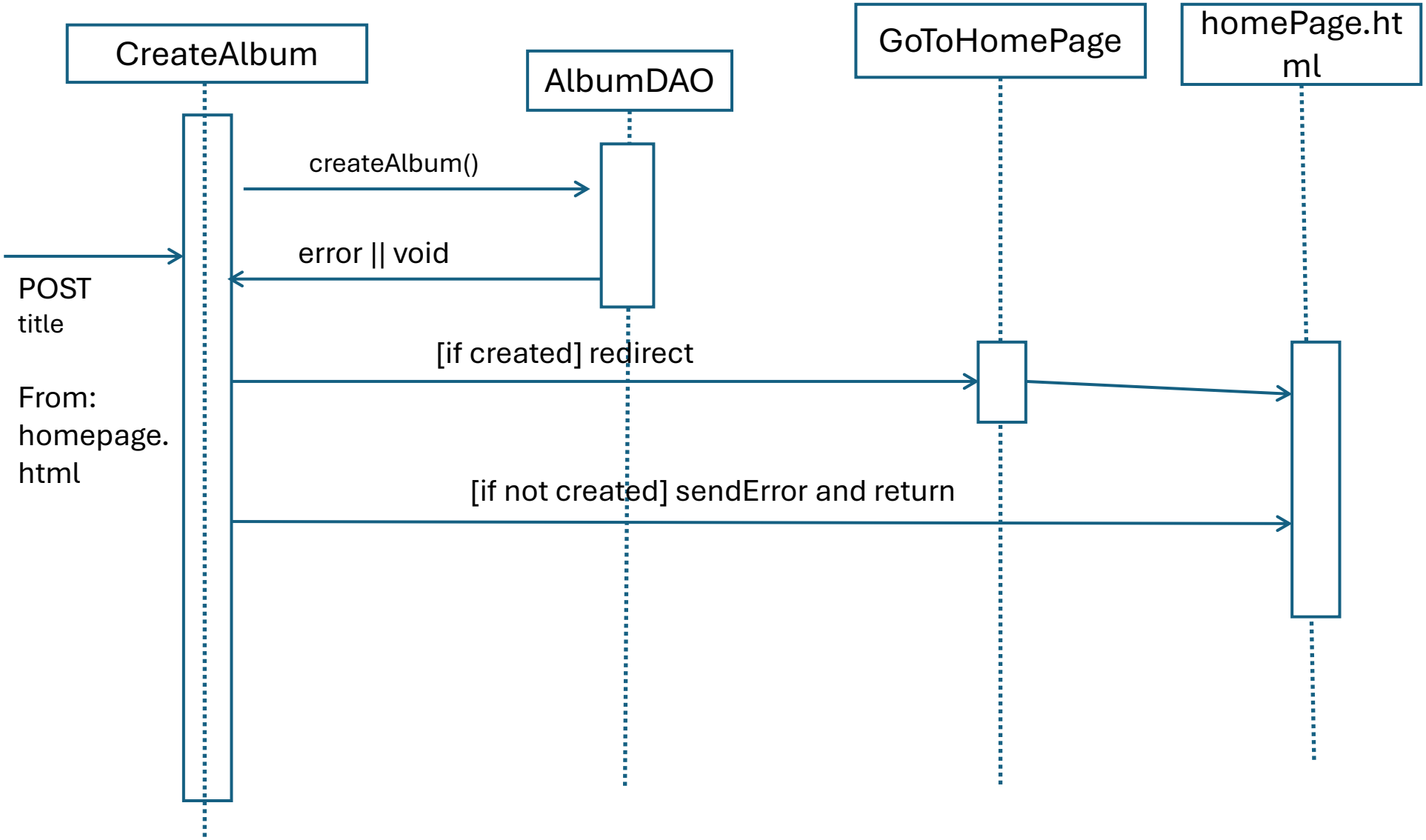
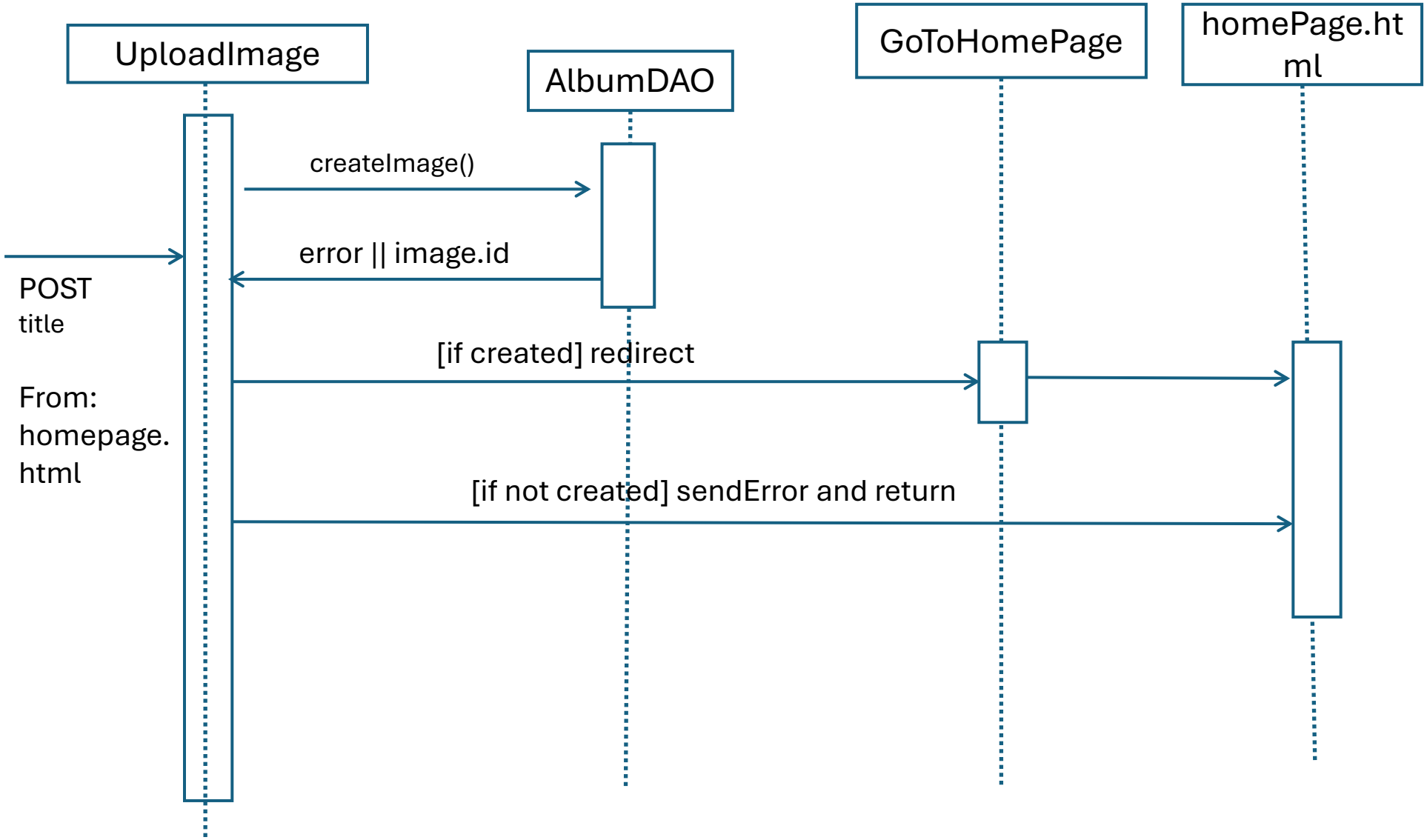
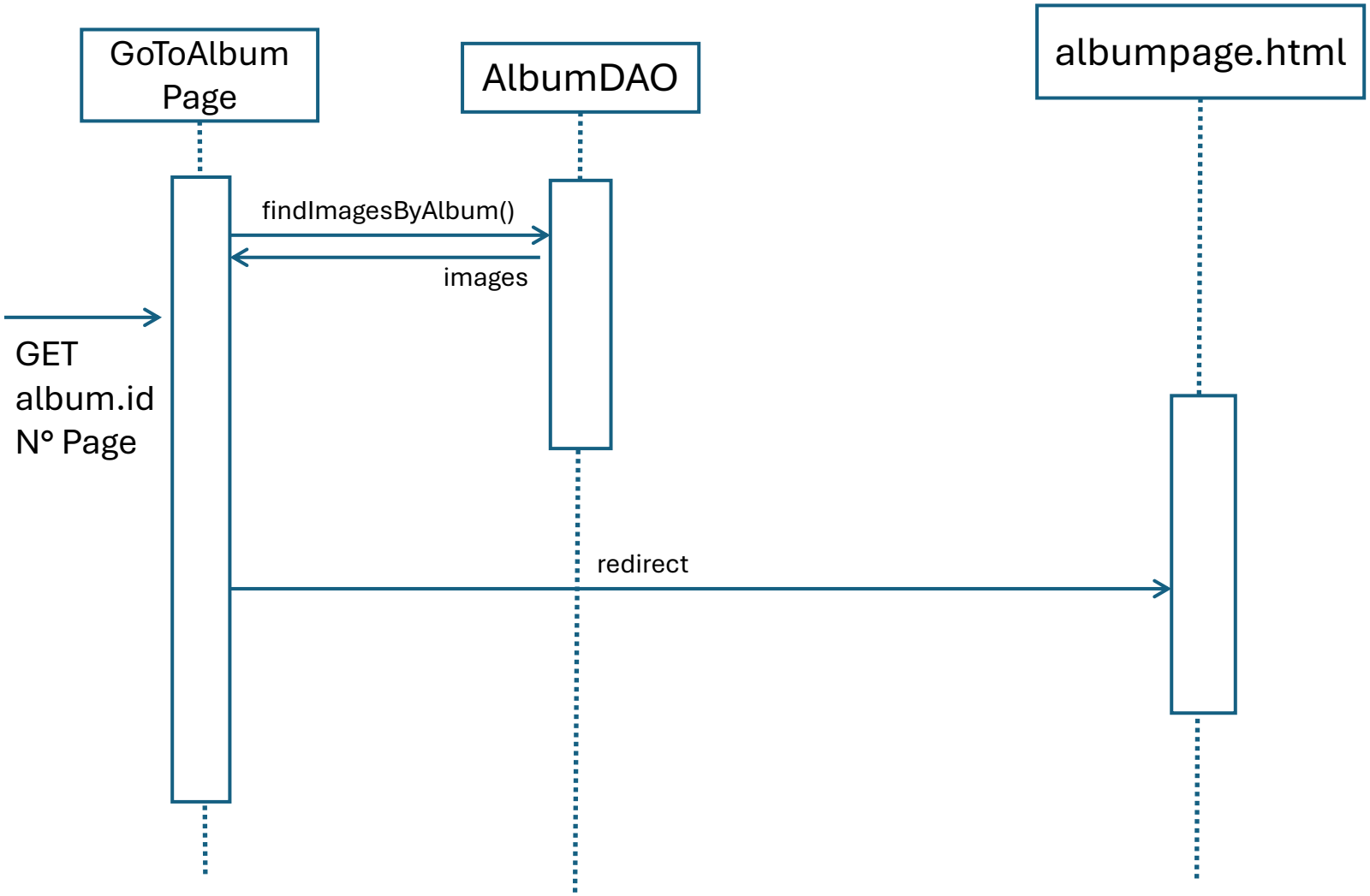
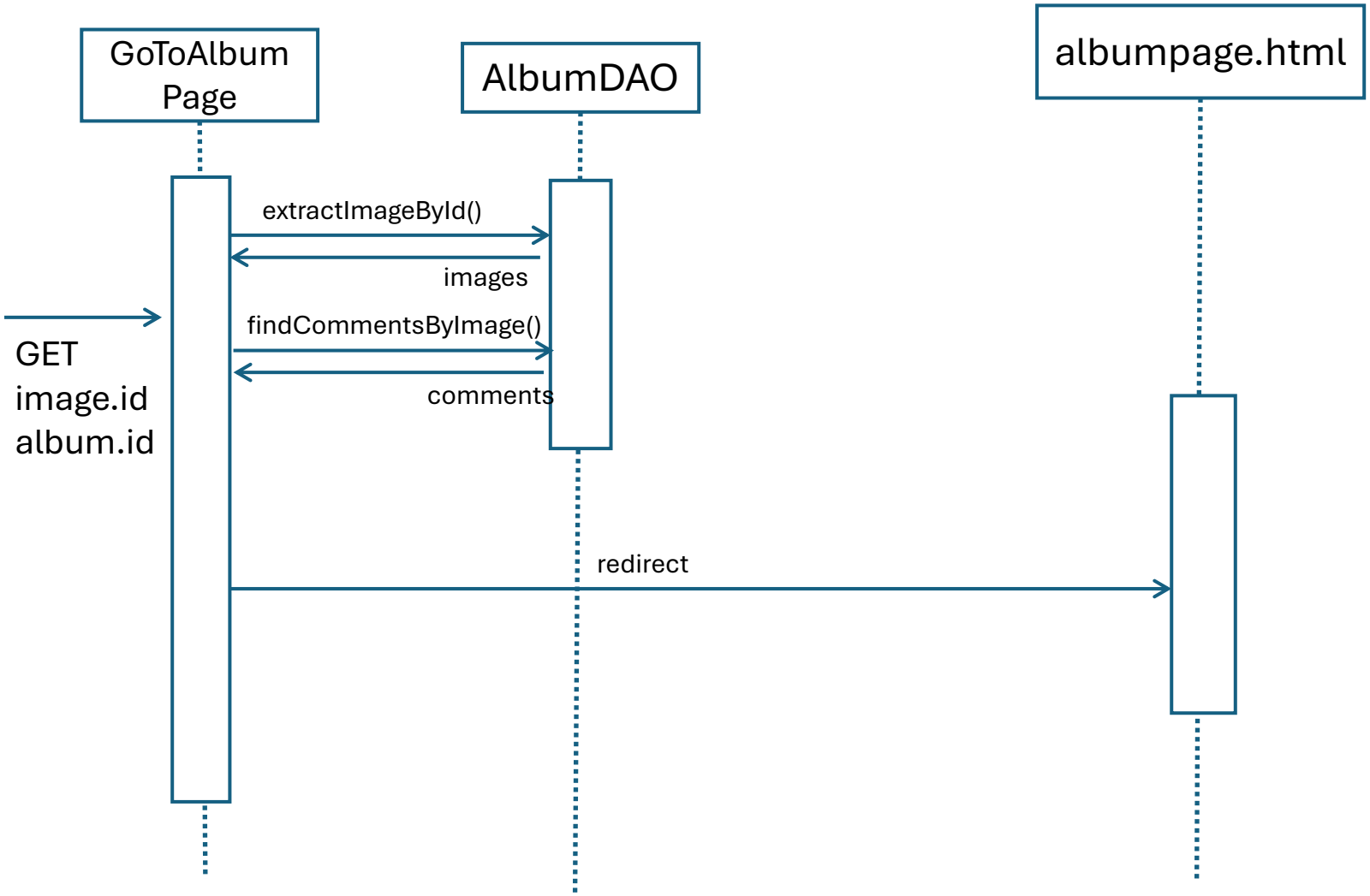
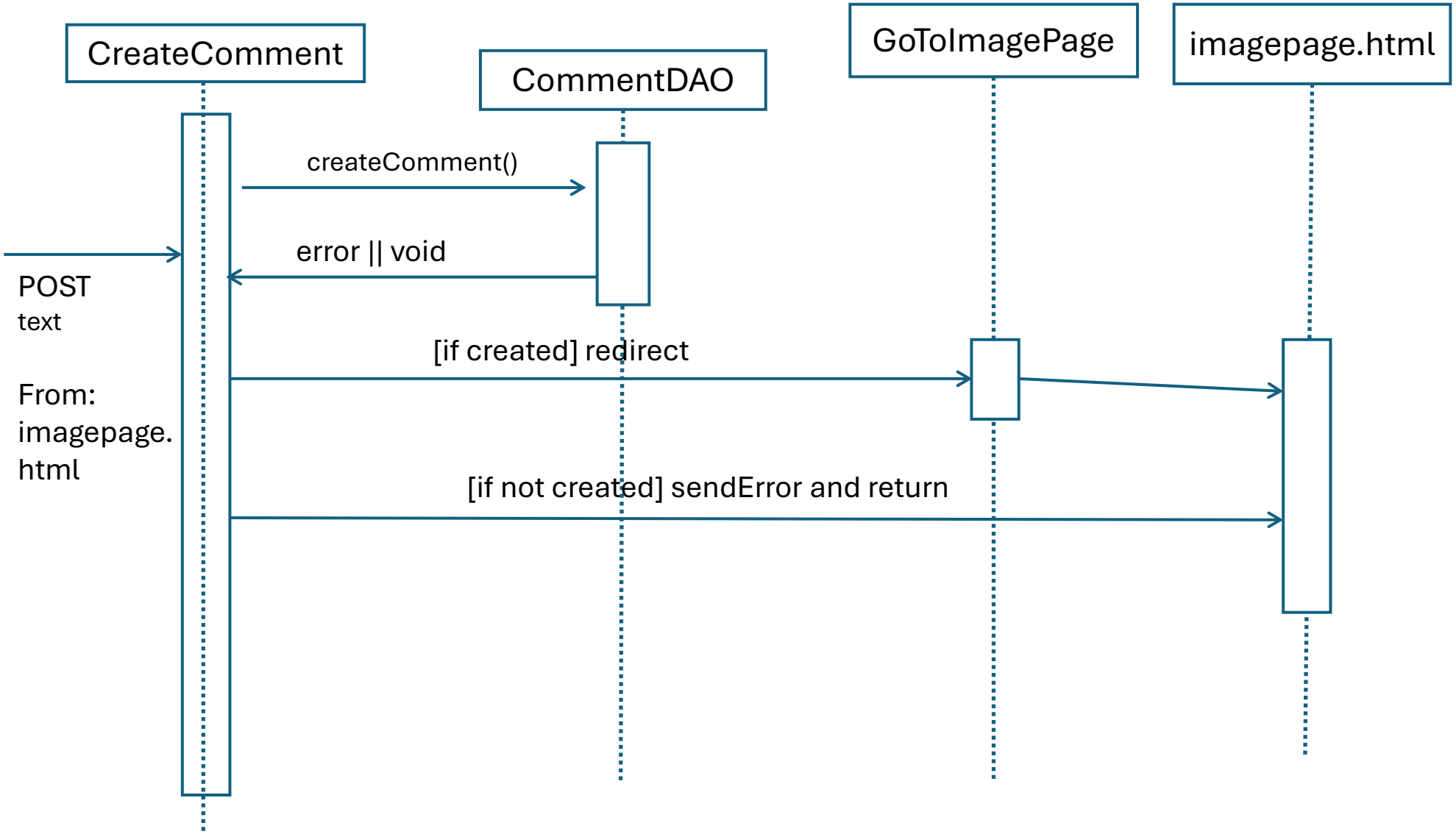
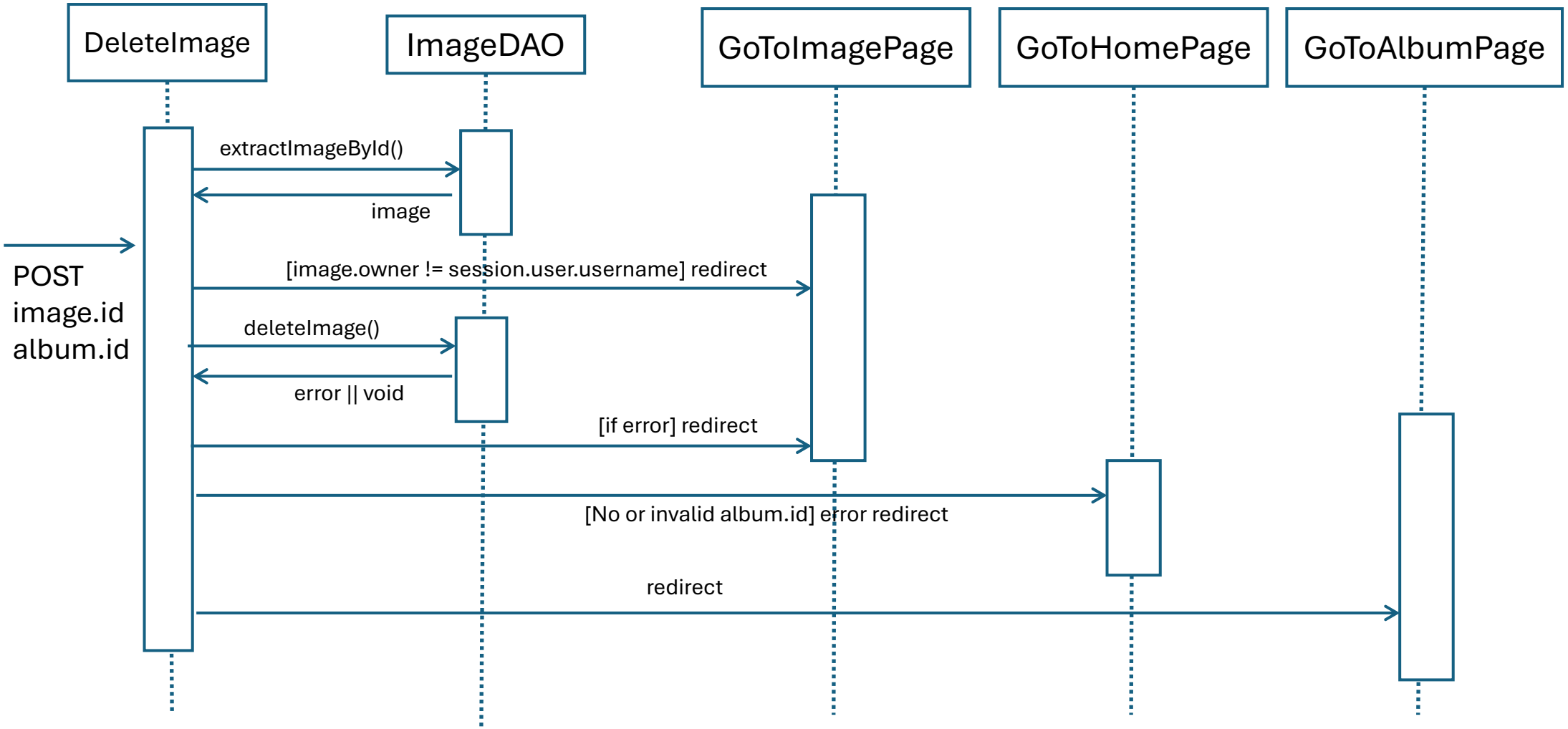
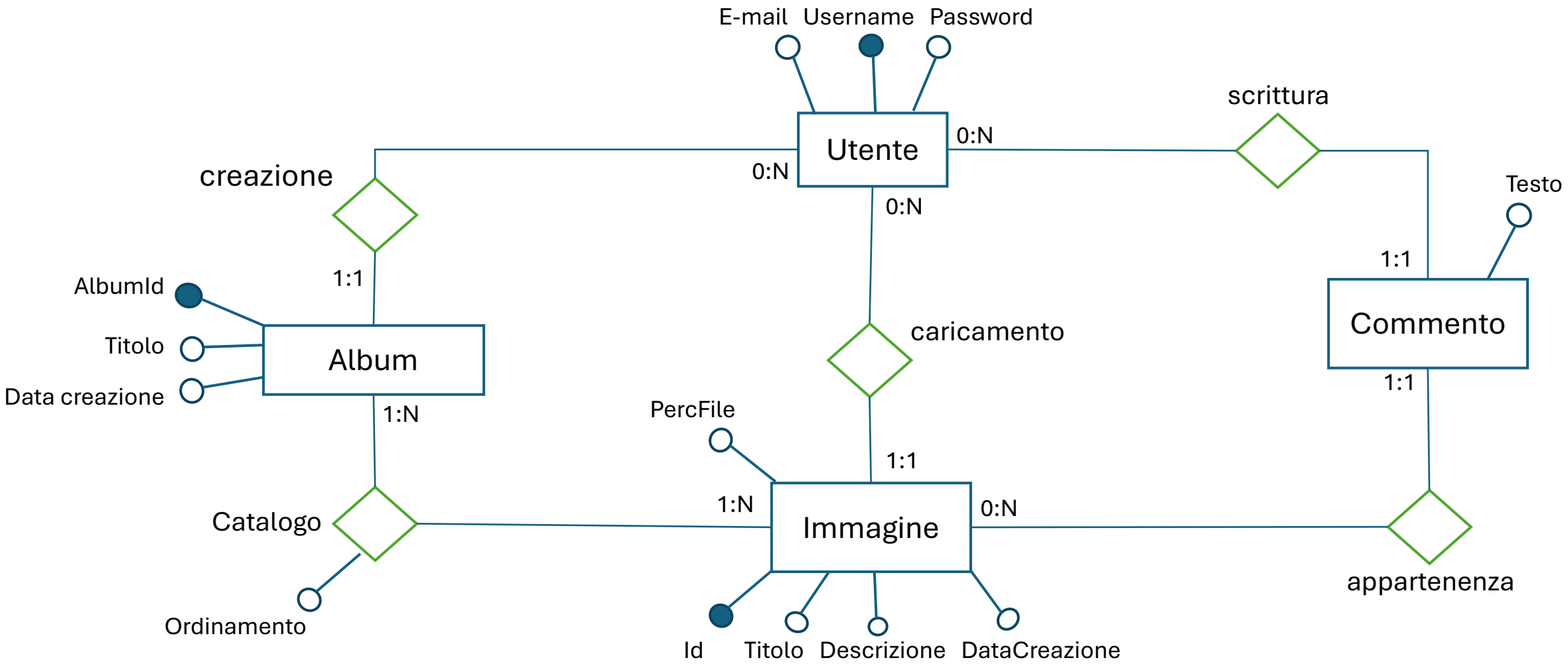
```
CREATE TABLE `users` (
`username` varchar(45) NOT NULL,
`password` varchar(45) NOT NULL,
`email` varchar(45) NOT NULL,
 PRIMARY KEY (`username`) )
```

```
CREATE TABLE `album` (
`albumId` int NOT NULL AUTO_INCREMENT,
`title` varchar(45) NOT NULL,
`creator` varchar(45) NOT NULL,
`creation_date` datetime NOT NULL,
PRIMARY KEY (`albumId`),
 CONSTRAINT `albumbyusr` FOREIGN KEY
(`creator`) REFERENCES `users`
(`username`) ON DELETE CASCADE )
```

```
CREATE TABLE `image` (
`imageId` int NOT NULL AUTO_INCREMENT,
`title` varchar(45) NOT NULL,
`description` varchar(45) NOT NULL,
`filepath` varchar(150) NOT NULL,
`owner` varchar(45) NOT NULL,
`creation_date` datetime NOT NULL,
PRIMARY KEY (`imageId`),
CONSTRAINT `imagebyusr` FOREIGN KEY
(`owner`) REFERENCES `users`
(`username`) ON DELETE CASCADE )
```

Local Database Schema

```
CREATE TABLE `comment` (
`id` int NOT NULL AUTO_INCREMENT,
`text` varchar(45) NOT NULL,
`creator` varchar(45) NOT NULL,
`imgId` int NOT NULL,
PRIMARY KEY (`id`),
CONSTRAINT `commentbyusr` FOREIGN
KEY (`creator`) REFERENCES `users`
(`username`) ON DELETE CASCADE,
CONSTRAINT `commentinimg` FOREIGN
KEY (`imgId`) REFERENCES `image`
(`imageId`) ON DELETE CASCADE )
```

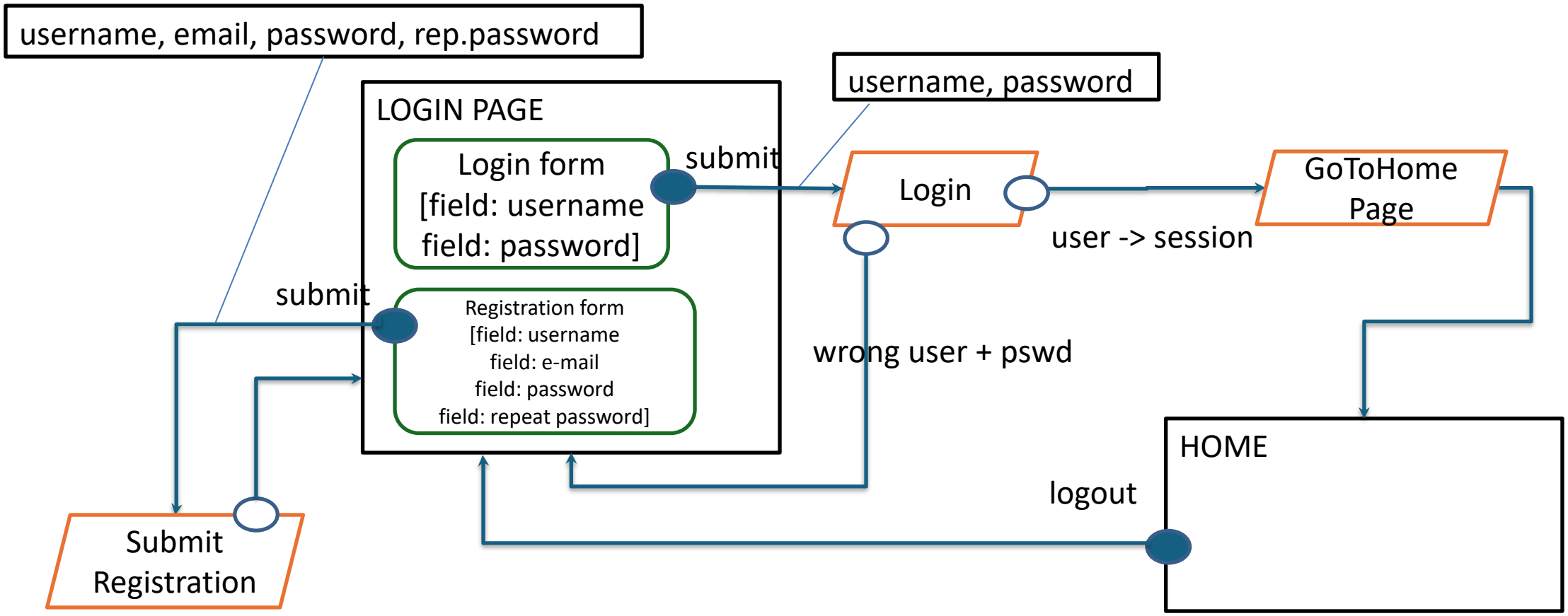
- ```
  CREATE TABLE `catalog` (
  `albId` int NOT NULL,
  `imgId` int NOT NULL,
  PRIMARY KEY (`albId`, `imgId`),
  CONSTRAINT `albuminctlg` FOREIGN KEY
  (`albId`) REFERENCES `album`
  (`albumId`) ON DELETE CASCADE,
  CONSTRAINT `imageinctlg` FOREIGN KEY
  (`imgId`) REFERENCES `image`
  (`imageId`) ON DELETE CASCADE )
  ```
  ```

Application Requirements Analysis

POLITECNICO
MILANO 1863

- Events: Registrazione all'applicazione, Accesso tramite Login, Creazione album, Accesso pagina Album, (Caricare Immagini), Accesso immagini successive, Accesso immagini precedenti, Accesso pagina dettaglio immagine, Creazione commento, Eliminazione immagine, Logout
- Pages (views): LOGIN PAGE, HOME PAGE, ALBUM PAGE, IMAGE PAGE
- Components: Elenco Album Personali (HOME), Elenco altri Album (HOME), Form creazione immagini (HOME), Form creazione album(HOME), Elenco Immagini Album (ALBUM),
  Form creazione commenti (IMAGE)
- Actions: Creazione Album, Creazione Immagine, Creazione Commento, Eliminazione Immagine

Application Design



Application Design



Application Design



Application Design



Components

- Controller (Servlets)
  1. LoginServlet
  2. SubmitRegistration
  3. GoToHomePage
  4. CreateAlbum
  5. UploadImage
  6. GoToAlbumPage
  7. GoToTimagePage
  8. CreateComment
  9. DeleteImage
  10. Logout
- View Components
  1. LOGIN PAGE : login form, registration form
  2. HOME PAGE: album creation form, upload image form, personal and non albums lists
  3. ALBUM PAGE: image list (partialized), button next and button previous
  4. IMAGE PAGE: comment creation form, comments list
- Model Objects (Beans)
  - User
  - Album
  - Image
  - Comment
- Data Access Objects
  - UserDAO
    - checkCredentials(), checkUniqueUsername(), checkUniqueMail()
    - createUser()
  - ImageDAO
    - createImage(), findImagesByAlbum(), extractImageById()
    - deleteImage()
  - AlbumDAO
    - findAlbumByUser(), findAlbumByOthers()
    - createAlbum()
  - CommentDAO
    - findCommentsByImage()
    - createComment()

Event: Login


LoginServlet
UserDAO
index.html
Session
GoToHome Page
checkCredentials
user || null
POST
username
password
[user == null] redirect
From:
index.
html
[user != null] setAttribute ("user", user)
[user != null] redirect

Event: Registration



Event: access HomePage



Event: Create Album



Event: Upload Image



Event: access AlbumPage



Event: access ImagePage



Event: Create Comment



Event: delete image



Progetto RIA

Data Design


E-mail
Username
Password
Utente
0:N
0:N
0:N
scrittura
creazione
1:1
AlbumId
Titolo
Data creazione
Album
1:N
caricamento
1:1
Testo
1:1
Commento
1:1
PercFile
1:N
Immagine
0:N
Catalogo
Ordinamento
appartenenza
Id
Titolo
Descrizione
DataCreazione

Local Database Schema

```
CREATE TABLE `users` (
`username` varchar(45) NOT NULL,
`password` varchar(45) NOT NULL,
`email` varchar(45) NOT NULL,
 PRIMARY KEY (`username`) )
```

```
CREATE TABLE `album` (
`albumId` int NOT NULL AUTO_INCREMENT,
`title` varchar(45) NOT NULL,
`creator` varchar(45) NOT NULL,
`creation_date` datetime NOT NULL,
PRIMARY KEY (`albumId`),
 CONSTRAINT `albumbyusr` FOREIGN KEY
(`creator`) REFERENCES `users`
(`username`) ON DELETE CASCADE )
```

```
CREATE TABLE `image` (
`imageId` int NOT NULL AUTO_INCREMENT,
`title` varchar(45) NOT NULL,
`description` varchar(45) NOT NULL,
`filepath` varchar(150) NOT NULL,
`owner` varchar(45) NOT NULL,
`creation_date` datetime NOT NULL,
PRIMARY KEY (`imageId`),
CONSTRAINT `imagebyusr` FOREIGN KEY
(`owner`) REFERENCES `users`
(`username`) ON DELETE CASCADE )
```

Local Database Schema

```
CREATE TABLE `comment` (
`id` int NOT NULL AUTO_INCREMENT,
`text` varchar(45) NOT NULL,
`creator` varchar(45) NOT NULL,
`imgId` int NOT NULL,
PRIMARY KEY (`id`),
CONSTRAINT `commentbyusr` FOREIGN
KEY (`creator`) REFERENCES `users`
(`username`) ON DELETE CASCADE,
CONSTRAINT `commentinimg` FOREIGN
KEY (`imgId`) REFERENCES `image`
(`imageId`) ON DELETE CASCADE )
```

- ```
  CREATE TABLE `catalog` (
  `albId` int NOT NULL,
  `imgId` int NOT NULL,
  `ord` int NOT NULL,
  PRIMARY KEY (`albId`, `imgId`),
  CONSTRAINT `albuminctlg` FOREIGN KEY
  (`albId`) REFERENCES `album`
  (`albumId`) ON DELETE CASCADE,
  CONSTRAINT `imageinctlg` FOREIGN KEY
  (`imgId`) REFERENCES `image`
  (`imageId`) ON DELETE CASCADE )
  ```
  ```

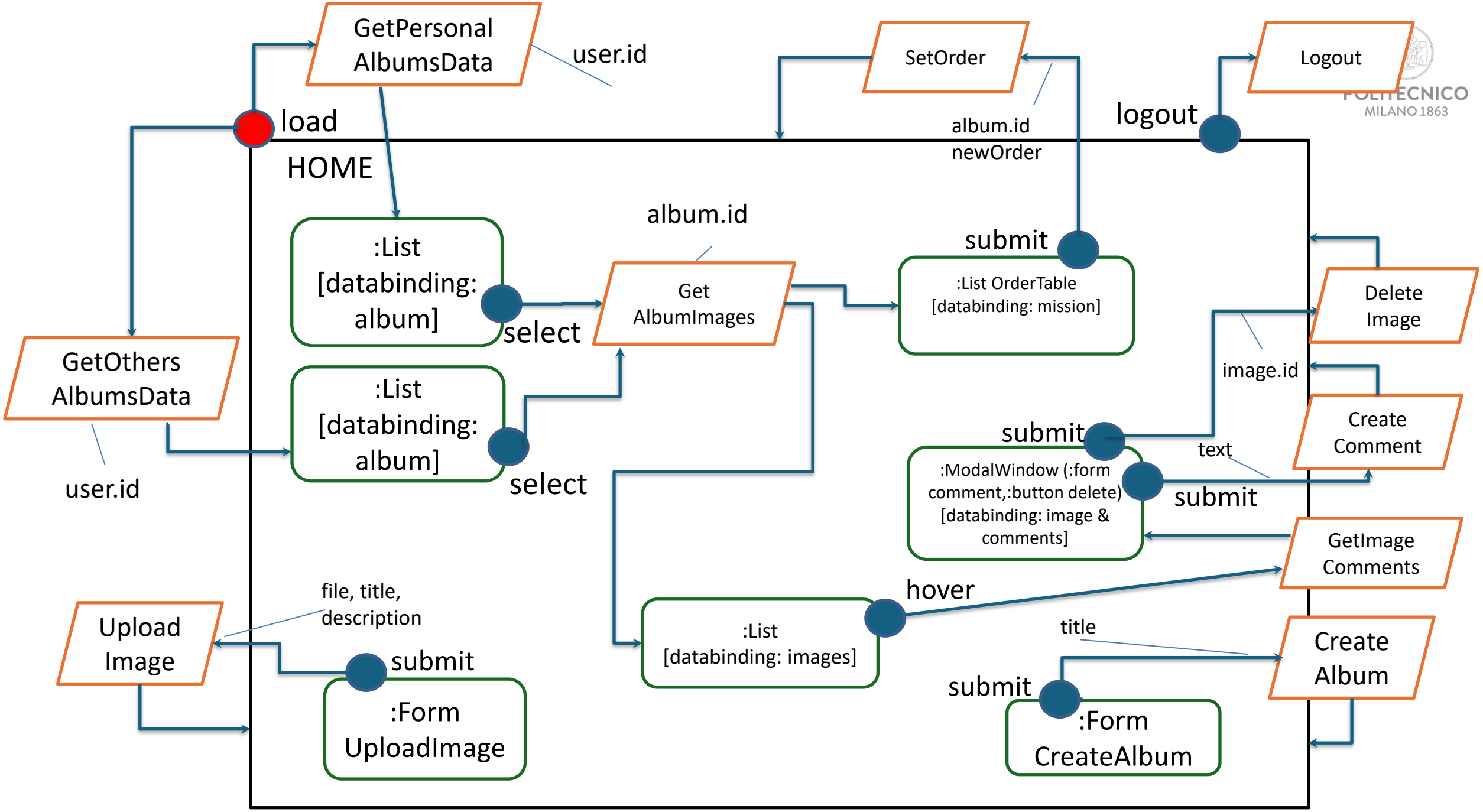
Application Requirements Analysis

POLITECNICO
MILANO 1863

- Events: Registrazione all'applicazione, Accesso tramite Login, Creazione album, Accesso pagina Album, (Caricare Immagini), Accesso immagini successive, Accesso immagini precedenti, Cambio ordine immagini, Accesso pagina dettaglio immagine, Creazione commento, Eliminazione immagine, Logout
- Pages (views): LOGIN PAGE, HOME
- Components: Elenco Album Personali (HOME), Elenco altri Album (HOME), Form creazione immagini (HOME), Form creazione album(HOME), Elenco Immagini Album (HOME),
  Form creazione commenti (HOME/ Modal Window), Elenco Ordine Immagini (HOME)
- Actions: Creazione Album, Creazione Immagine, Creazione Commento, Eliminazione Immagine, Cambio Ordine Immagini

Application Design




GetPersonal AlbumsData
user.id
SetOrder
Logout
load
HOME
album.id
newOrder
logout
:List [databinding: album]
select
album.id
Get AlbumImages
submit
:List OrderTable [databinding: mission]
Delete Image
GetOthers AlbumsData
user.id
:List [databinding: album]
select
image.id
Create Comment
submit
text
:ModalWindow (:form comment,:button delete) [databinding: image & comments]
submit
GetImage Comments
hover
file, title, description
Upload Image
:List [databinding: images]
title
Create Album
submit
:Form UploadImage
submit
:Form CreateAlbum

Components

- Controller (Servlets)
  1. Login
  2. SubmitRegistration
  3. GetPersonaAlbumsData
  4. GetOthersAlbumsData
  5. CreateAlbum
  6. UploadImage
  7. GetAlbumImages
  8. GetImageComments
  9. SetOrder
  10. CreateComment
  11. DeleteImage
  12. Logout
- View Components
  1. LOGIN PAGE : login form, registration form
  2. HOME: album creation form, upload image form, personal and non albums lists, image list (partialized), button next and button previous, order list, comment creation form, comments list
- Model Objects (Beans)
  - User
  - Album
  - Image
  - Comment
- Data Access Objects
  - UserDAO
    - checkCredentials(), checkUniqueUsername(), checkUniqueMail()
    - createUser()
  - ImageDAO
    - createImage(), findImagesByAlbum(), extractImageById()
    - deleteImage(), setOrder()
  - AlbumDAO
    - findAlbumByUser(), findAlbumByOthers()
    - createAlbum()
  - CommentDAO
    - findCommentsByImage()
    - createComment()

Client side: view & view component

- Index
  - Login form
    - Gestione del submit e degli errori
  - Registration form
    - Gestione del submit e degli errori
- Home
  - CreateAlbum
    - show(): rende visibile l'oggetto
    - registerEvents(): associa al componente le funzioni per gestirne gli eventi
    - reset(): ripristina la visibilità iniziale
  - UploadImage
    - registerEvents(): associa al componente le funzioni per gestirne gli eventi
    - show(): rende visibile l'oggetto
    - update(): riceve dati da altri oggetti e aggiorna dinamicamente sotto componenti
    - reset(): imposta le condizioni di iniziali visibilità dei vari sotto-componenti
  - AlbumList (myAlbumList & otherAlbumList)
    - show(): riceve dati dal server da mostrare
    - reset(): imposta le condizioni di iniziale visibilità dei vari sotto-componenti
    - update(): aggiorna dinamicamente l'oggetto da mostrare

Client side: view & view component

- Home
  - ImagesAlbum
    - show(): riceve dati dal server e li gestisce per visualizzarli
    - registerEvents(): associa al componente le funzioni per gestirne gli eventi
    - reset(): ripristina la visibilità iniziale sua e dei vari sotto-componenti
    - update(): aggirona dinamicamente l'oggetto e le informazioni da mostrare
  - OrderImages
    - registerEvents(): associa al componente le funzioni per gestirne gli eventi
    - show(): gestisce le informazioni scaricate da altri oggetti e visualizza l'oggetto e i sotto-componenti
    - reset(): imposta le condizioni di iniziali visibilità dei vari sotto-componenti
  - ImageDetails
    - show(): riceve dati dal server da mostrare
    - registerEvents(): associa al componente le funzioni per gestirne gli eventi
    - reset(): imposta le condizioni di iniziale visibilità dei vari sotto-componenti
    - update(): aggiorna dinamicamente l'oggetto da mostrare
    - close(): chiude la finestra modale
  - PageController
    - start(): inizializza gli oggetti
    - refresh(): imposta gli oggetti alle condizioni iniziali